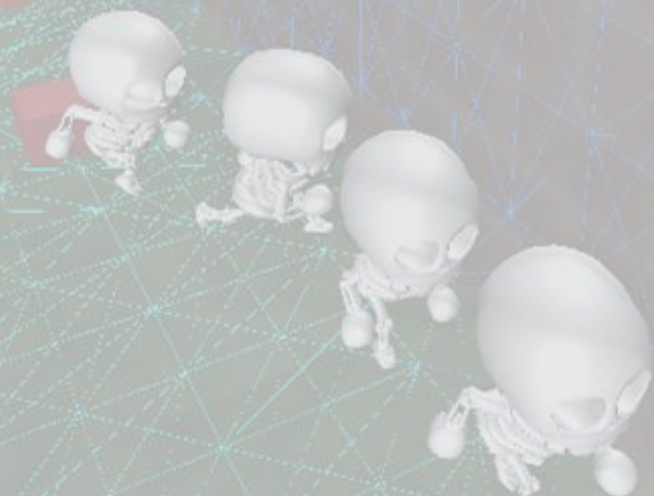


ポートフォリオ



学校法人情報文化学園アーツカレッジヨコハマ

ゲームクリエイター学科プログラマーコース 3年 君嶋 宏之

目次

自己PR

作品概要

- ・ ゲームについて
- ・ システムについて

ナビメッシュの構築と制御

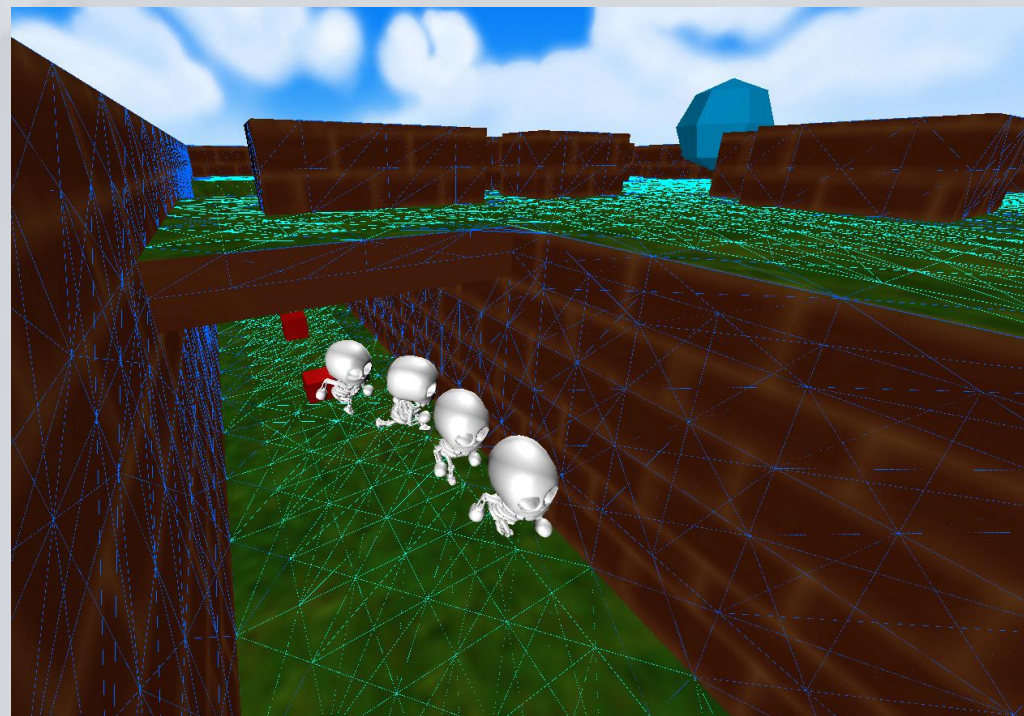
- ・ ナビメッシュの構築
- ・ 自然な移動をするための工夫

敵ルートのコスト算出

処理負荷軽減の取り組み

- ・ 設置時のみのOBB当たり判定
- ・ 再計算をしなくていいようにする工夫

JSONを使ったデータの外部化



自己PR

好きなゲーム作品



強み

探求心

ナビゲーションAIをゲームに導入するにあたり、授業内では得られなかった知識を主体的に調べ学習し、ナビメッシュやボクセルでのノード分割などを自身のゲーム制作に導入しました。調べ方もインターネットや本を活用し、情報源の幅を広げて知識を深めていきました。

粘り強さ

身につけた知識を積極的に作品に取り入れ、失敗を恐れず繰り返しゲーム制作をし続ける粘り強さがあります。放課後の数時間や休日を使いナビゲーションAIについて調べ、新しい知識としてD*Liteや高速ナビゲーション、空中での高度なナビゲーションAIなどを学びました。これらを積極的にゲーム制作に導入し、失敗からも学びを得て、よりよいゲームを制作するために努力しています。

スキル

プログラミング

- ・ C、C++、C#
- ・ Unity
- ・ TortoiseSVN
- ・ Visual Studio



その他

- ・ Blender
- ・ Word
- ・ Excel
- ・ PowerPoint
- ・ Photoshop
- ・ Premiere Pro

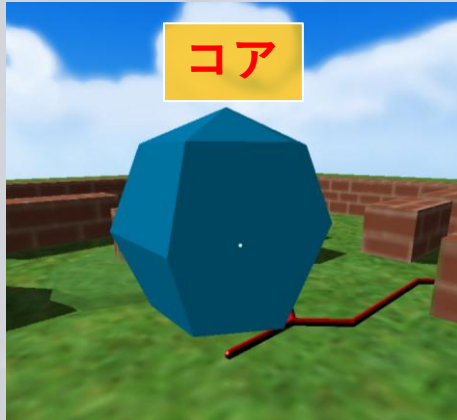


作品概要

作品概要 [ゲームについて]

ゲーム目的

- ・ 敵から**コア**を防衛することです。



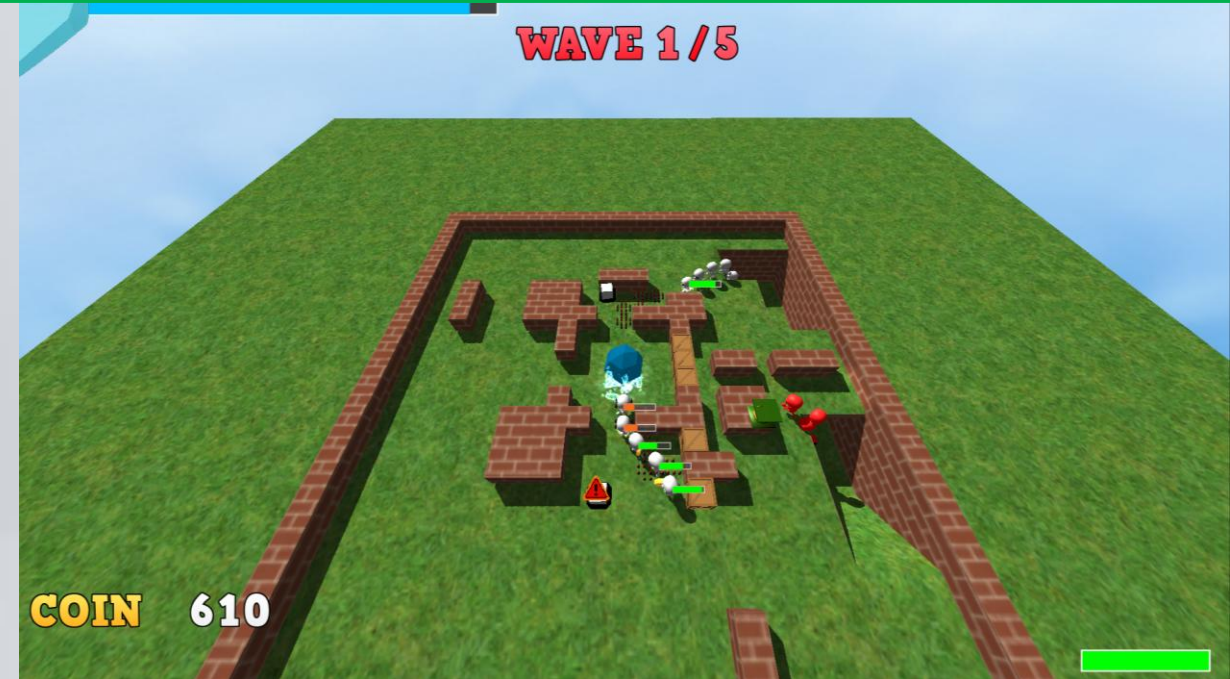
防衛方法

<一人称視点>

- ・ プレイヤーが敵を妨害できます。

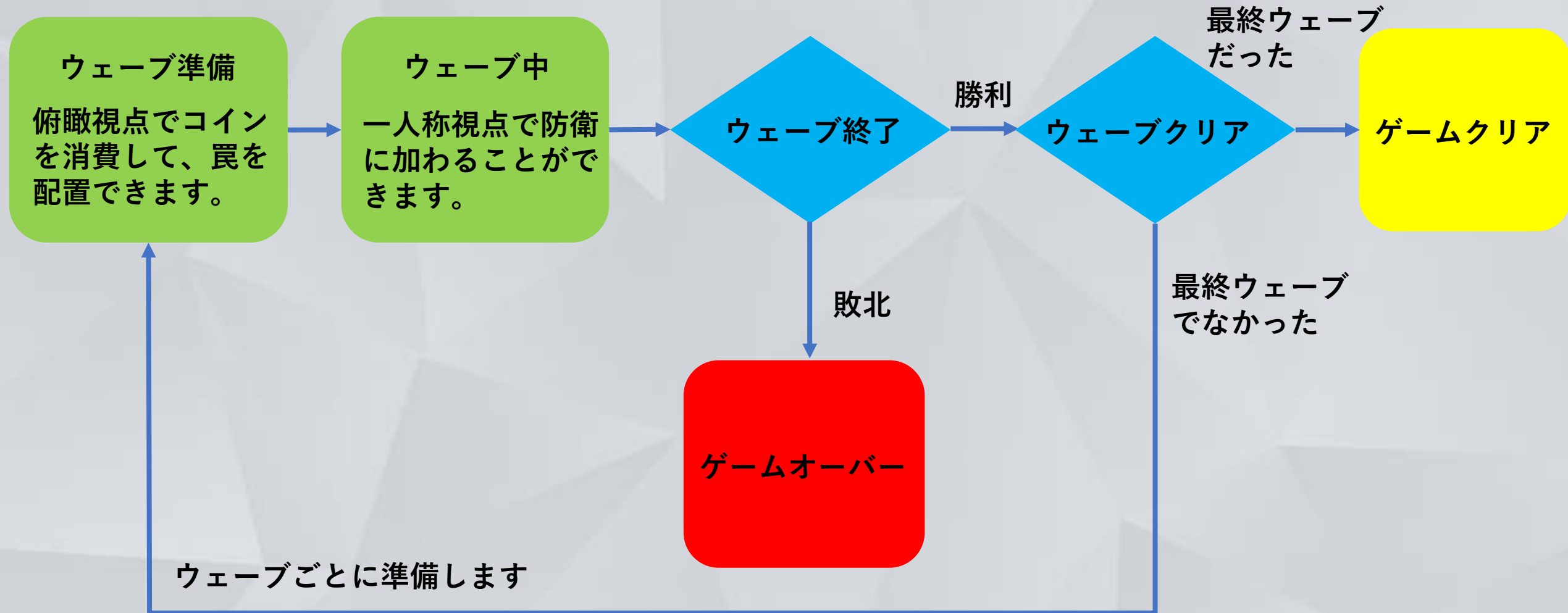
<俯瞰視点>

- ・ 罠を設置できます。



作品概要 [ゲームについて]

ゲームの流れ



作品概要 [ゲームについて]

ウェーブ準備

俯瞰視点で設置できるもの

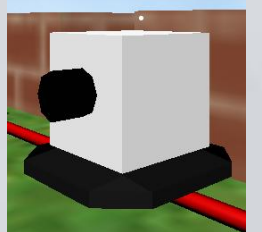
壁

- ・ 敵のルートを遮断します。



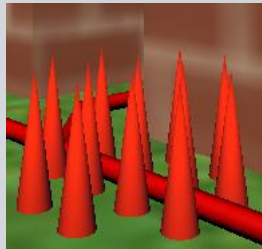
タレット

- ・ 敵に弾を発射してダメージを与えます。



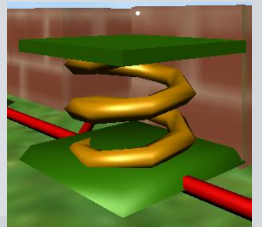
スパイク

- ・ 触れた敵にダメージを与えます。



ジャンプパッド

- ・ 触れたプレイヤーの移動を加速させます。

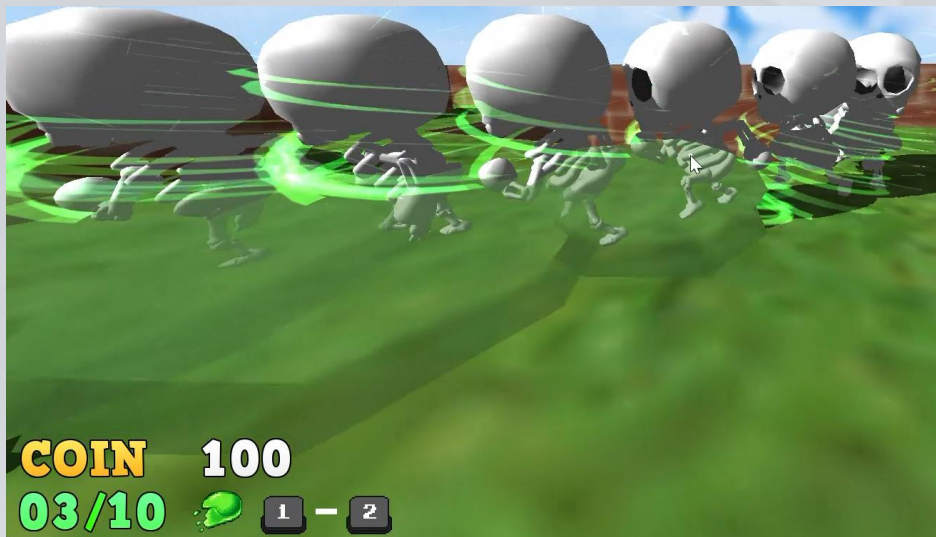


- ・ 罠の設置をするだけでなく、一人称視点でプレイヤーも防衛に介入できるようにすることで、**自身も戦える臨場感のあるゲーム体験**を実現しました。

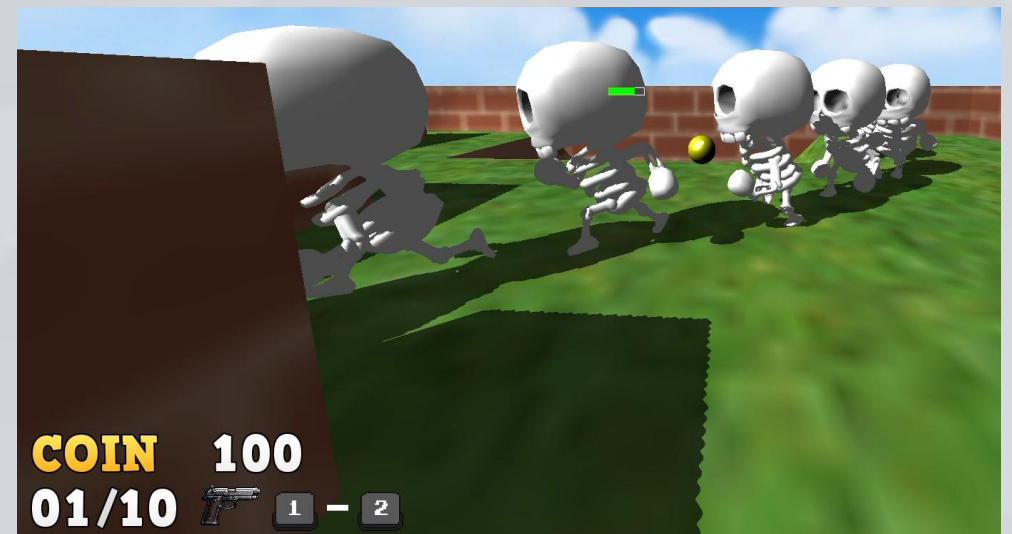
作品概要 [ゲームについて]

ウェーブ中

一人称視点での発射物



- ・ピストルで弾を発射して敵への攻撃ができます。

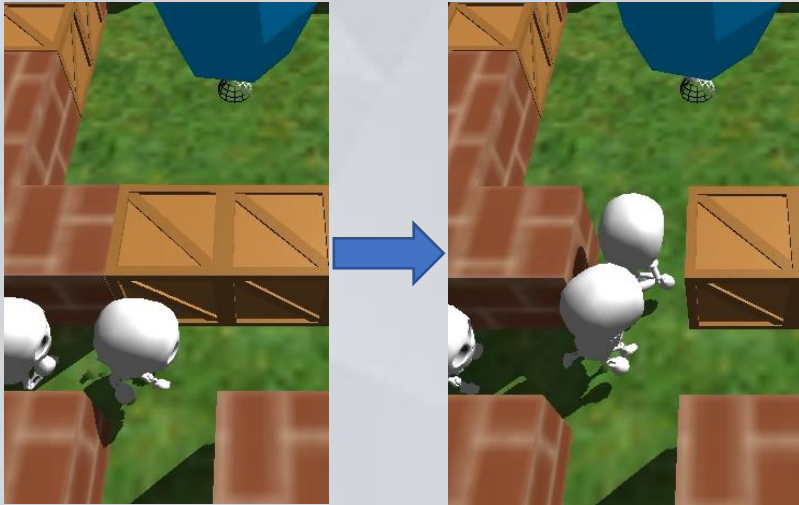


- ・スライムを発射して触れた敵の行動を遅くできます。

作品概要 [ゲームについて]

敵の行動

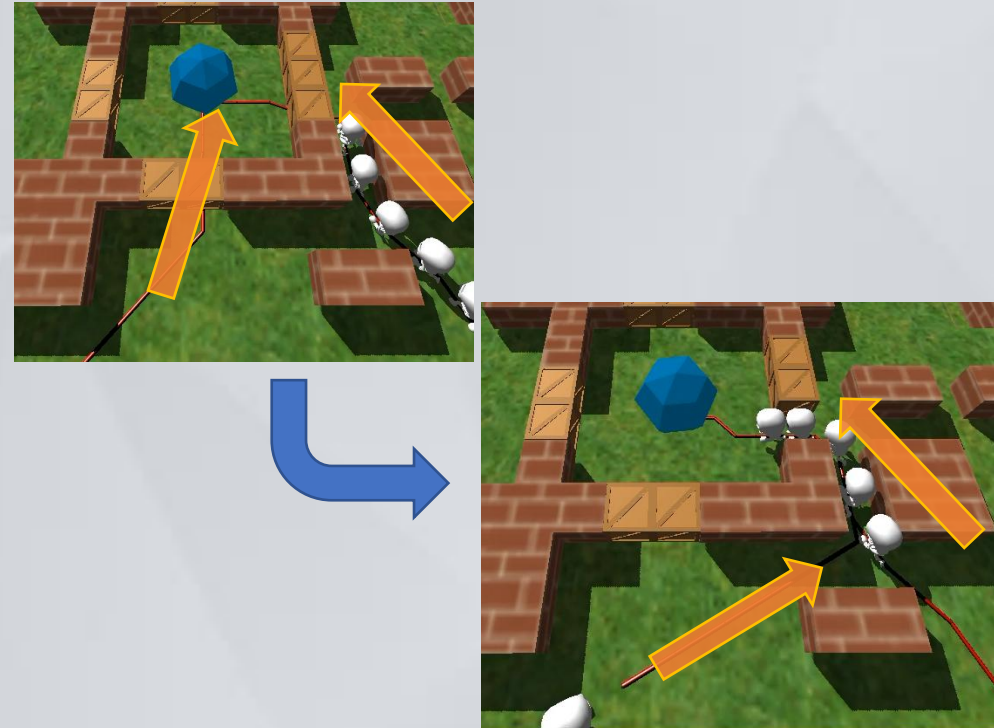
壁を破壊する



壁を登る



ルートの動的変更

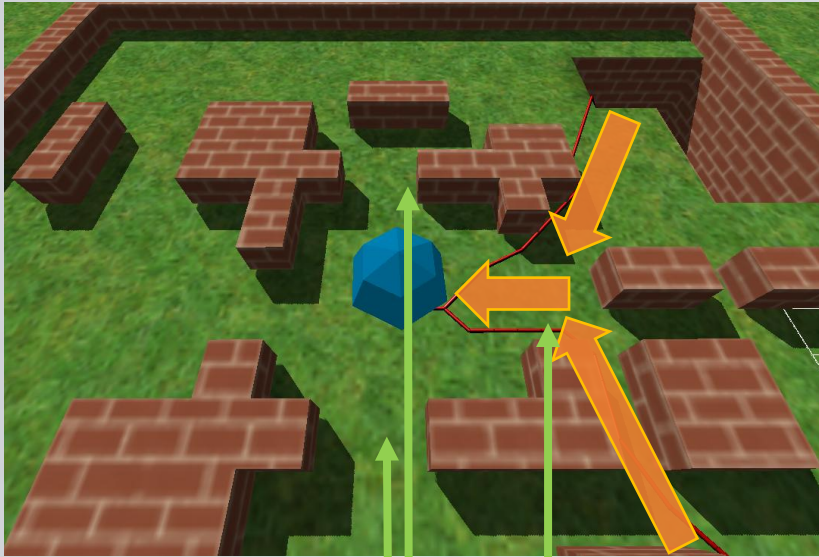


- ・ 様々な敵の行動を導入することによって、敵の経路の複数化と変化を導入して、**常に変化する緊張感のあるゲーム体験**を実現しました。

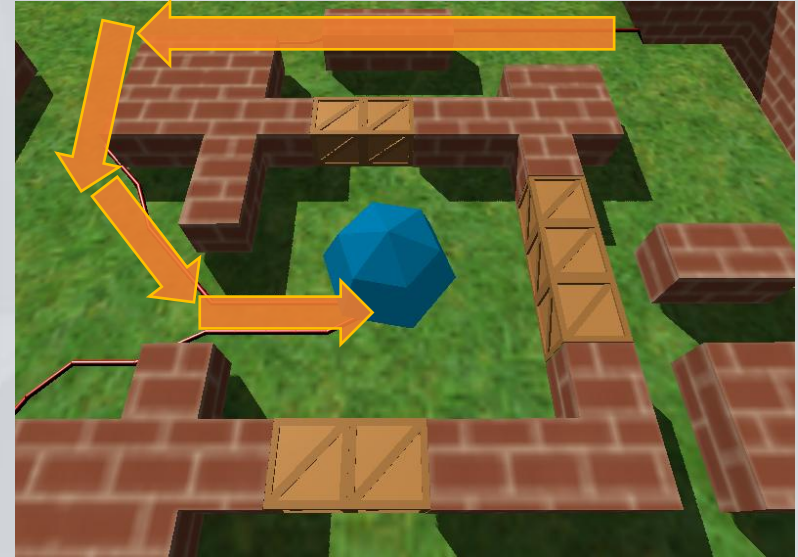
作品概要 [システムについて]

動的ナビゲーションの導入

ステージ変化前



ステージ変化後



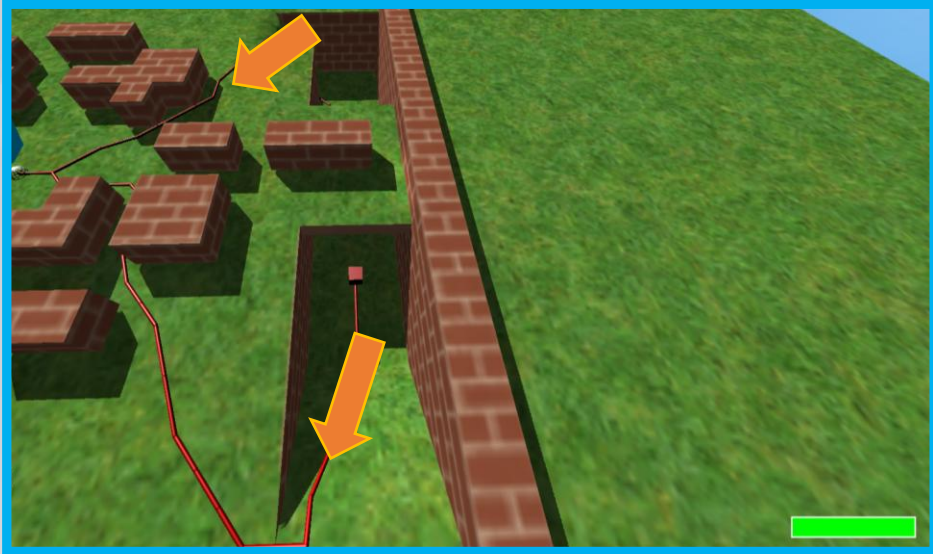
この場所にオブジェクトを設置

- ・ 敵の経路に罠を置くと動的に経路が変化します。
- ・ 設置、削除されたオブジェクトの位置や形状から、ナビゲーションのWayPointの使用フラグをON・OFFすることで、経路の変更を実現しました。

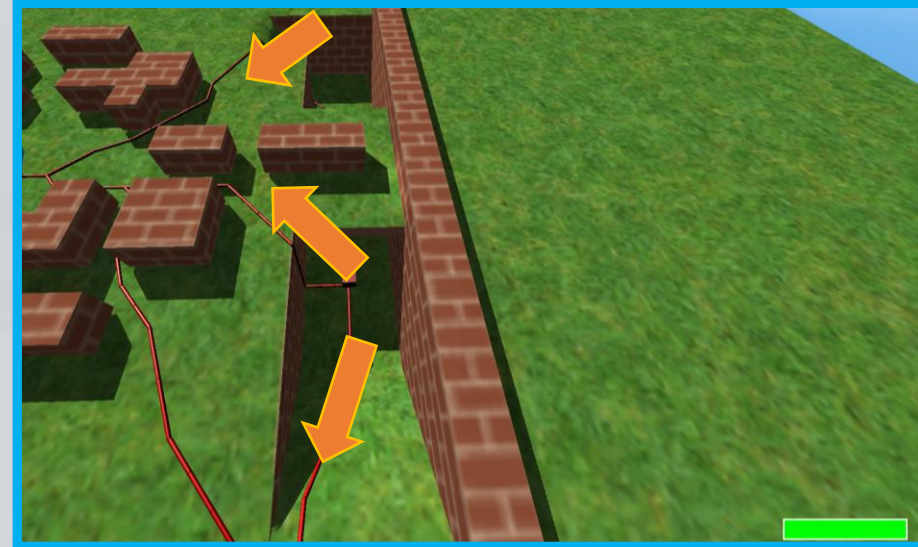
ナビメッシュの構築と制御

ナビメッシュの構築と制御 [ナビメッシュの構築]

壁を登る敵を導入する前

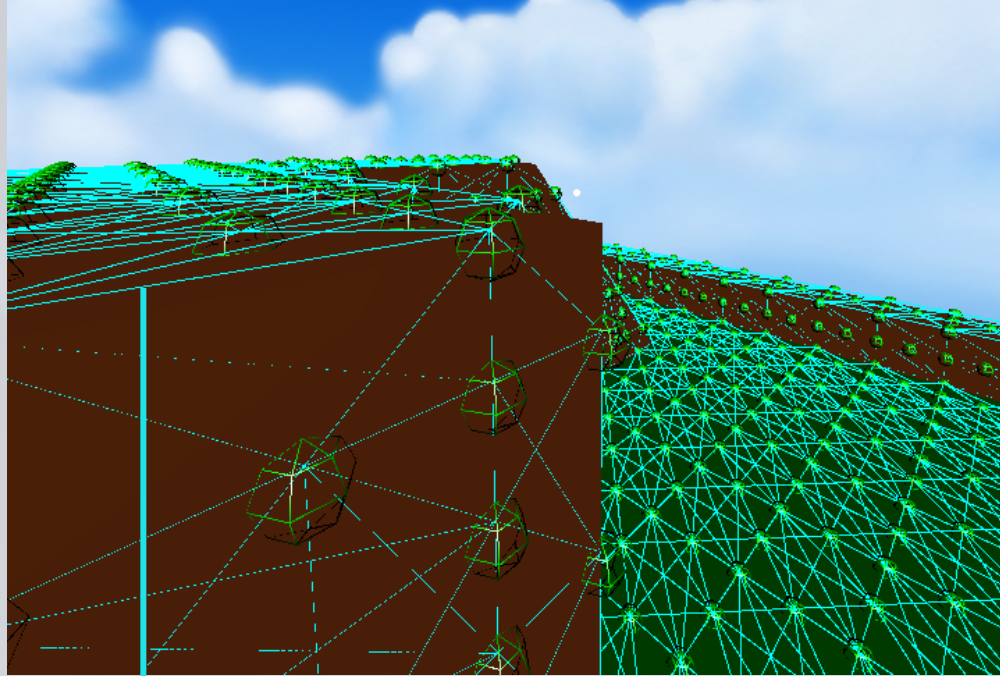


壁を登る敵を導入した後

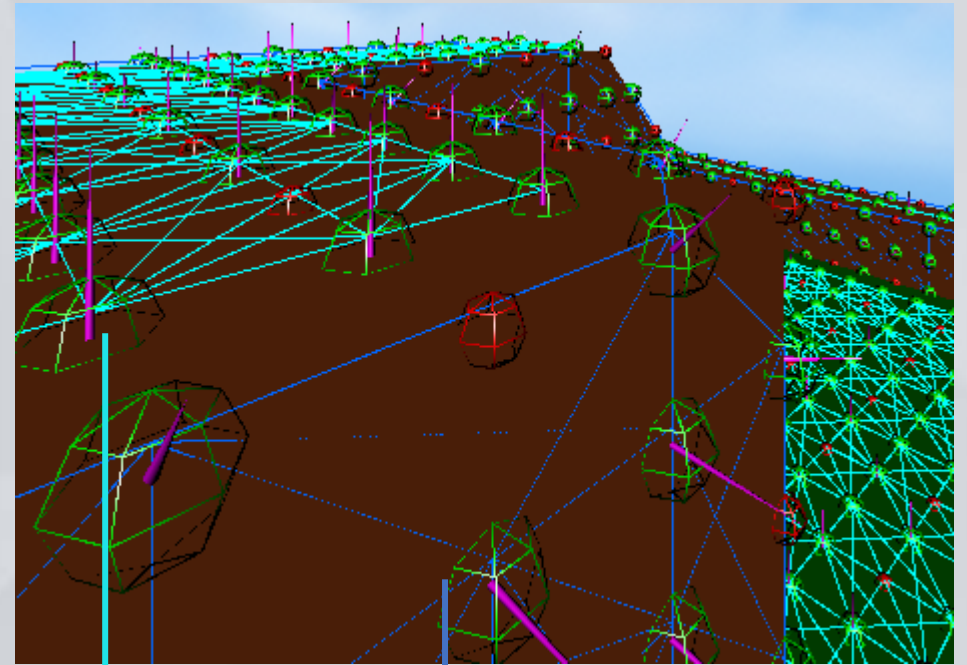
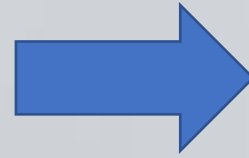


- ・ 敵の進行ルートのパターンが単一になりやすかったので、**壁登りを導入することで、進行ルートを複数化**し、ユーザーの対応力が問われる、緊迫感のあるゲーム体験を導入しました。

ナビメッシュの構築と制御 [ナビメッシュの構築]



通常ノード



通常ノード

壁ノード

- ・ モデルの法線方向でノードの種類を選定します。
 - > 壁を登る敵が使うノードの選定を行います。
 - > 不必要なノードの選定もここで行っています。

ナビメッシュの構築と制御 [ナビメッシュの構築]

実際のソース

- 1 ポータル^[補足]を構築するポリゴンの法線値を求めます。
- 2 法線値とYプラス方向の法線値の差の値によってノードを以下のように種類分けします。

- > 差が60度以上 : 壁ノード
- > 差が135度以上 : ノードを除外
- > それ以外 : 通常ノード

Yプラス方向の法線値



```
// 平均法線を算出
VECTOR3 averageNorm = VDivI(sumNorm, (int)normCon.size());
// 平均法線を設定
por.SetPortalDataAverageNorm(averageNorm);

// 法線の値と0,1,0の方向ベクトルで内積をとる
float dot = VDot(VECTOR3(0, 1, 0), VECTOR3(0, minNormY, 0));

// 60度以上の差があった場合
if (dot <= cosf(WALL_SET_NORM_ANGLE))
{
    // 135度を越えた差があった場合 立方体の角は含まない程度の法線ベクトル
    // 底面は追加しない(ナビゲーションしないので)
    if (dot < cosf(135.0f * DegToRad))
        continue;

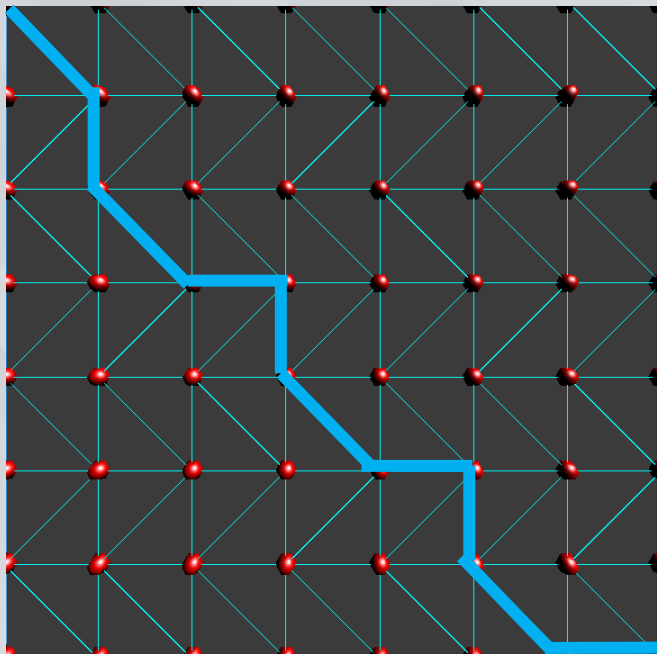
    // 壁のNavPointと設定する
    por.SetUseKind(NODE_USE_KIND::WALL);
}
else
{
    // ノーマルのNavPointと設定する
    por.SetUseKind(NODE_USE_KIND::NORMAL);
}
```

※ ポータル ポリゴン同士が共有するエッジのことです。

ナビメッシュの構築と制御 [自然な移動をするための工夫]

頂点接続情報に基づいたナビメッシュ

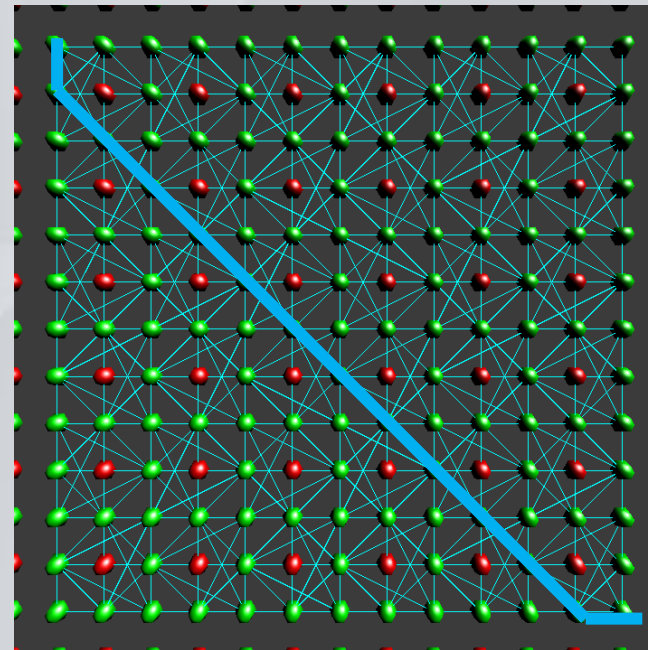
start



goal

ポータルの中間点に基づいたナビメッシュ

start



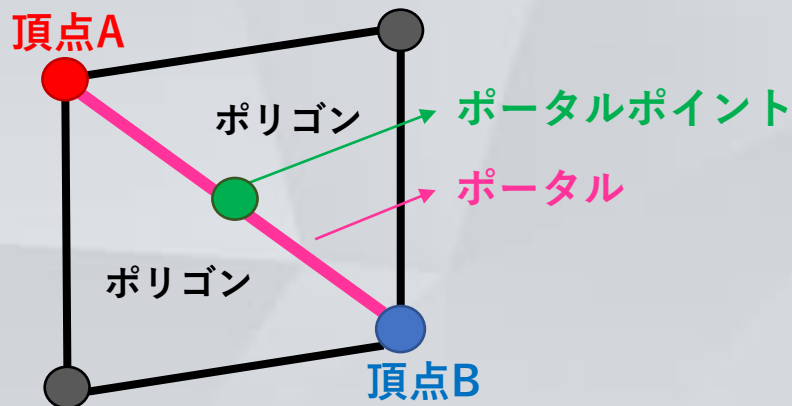
goal

- ・ポータルの中間点を接続してナビメッシュを構築することで、より滑らかな、ポリゴンの境界や形を感じさせない自然な経路を構築できるようにしました。

ナビメッシュの構築と制御 [自然な移動をするための工夫]

実際のソース

- 1 頂点を保存した配列を回し同じポリゴンを共有した頂点同士(A,B)を探します。
- 2 頂点同士の座標の平均値からポータル^{ポータル}の中心座標を求め配置します。
- 3 中心座標を使ってポータル^{ポータル}ポイント(ノード)の生成を行います。



```
int fNum = vData.number; // 小さい方の頂点ナンバー A
int sNum = vTarData.number; // 大きい方の頂点ナンバー B

// 小さい方の頂点ナンバーが大きい方の頂点ナンバーよりも大きかったら
// ペア a,b == b,aとするハッシュを上手く動かすため
if (fNum > sNum)
    std::swap(fNum, sNum);

std::pair<std::unordered_set<std::pair<int, int>, SymmetricPairHash>::iterator, bool> isPush =
    portalList.insert(std::make_pair(fNum, sNum));

// 追加することができたら 新しい組み合わせペアだったら
if (isPush.second)
{
    // 2点間座標の平均
    VECTOR3 portalCenterPos = (vData.position + vTarData.position) / 2.0f; // ポータルの中心座標

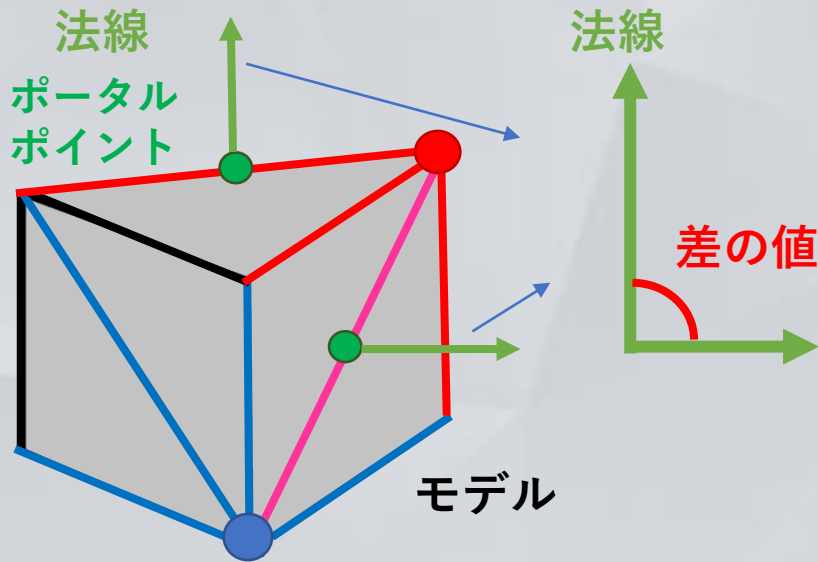
    // ポータルポイントコンテナに生成ナンバーとクラスをpush
    modelForLocalPortalPointList[blockKind].insert(std::make_pair(
        createNum,
        PortalPoint(
            createNum,
            portalCenterPos,
            PortalData(fNum, sNum),
            OwnerModelData(NavMeshController::ownerModelSaveNum, polData.MaxPosition * blockTrans.GetMatrix(), b)
        )
    ));
}
```

ナビメッシュの構築と制御 [自然な移動をするための工夫]

実際のソース

- 4 生成したポータルポイントを構築する頂点(A,B)がもつポータルポイント同士の法線を求めます。
- 5 法線の差の値を見て以下のように接続するか判断します。

- > 差がない 接続する
- > 差がある 接続しない



```
std::vector<VECTOR3> trPorNormCon = GetPortalPointShaerdPolygonNorm(trPor); // 相手のポータルの頂点ABで構築されるポリゴンの法線のコンテナを返す
std::vector<VECTOR3> mePorNormCon = GetPortalPointShaerdPolygonNorm(mePor); // 自身のポータルの頂点ABで構築されるポリゴンの法線のコンテナを返す

for (auto trNormItr = trPorNormCon.begin(); trNormItr != trPorNormCon.end(); trNormItr++)
{
    for (auto meNormItr = mePorNormCon.begin(); meNormItr != mePorNormCon.end(); meNormItr++)
    {
        VECTOR3 trNorm = *trNormItr;
        VECTOR3 meNorm = *meNormItr;

        constexpr float SAME_DOT = 0.999f; // floatの誤差を考慮した同じベクトル方向の内積の判定数値

        // 同じ法線方向だったら
        if (VDot(trNorm, meNorm) >= SAME_DOT)
        {
            doConnect = true; // 接続するフラグをオン

            VECTOR3 mePorPos = mePor.GetPosition(); // 自身のポータルポイントの座標
            VECTOR3 trPorPos = trPor.GetPosition(); // 相手のポータルポイントの座標

            break;
        }
    }
}

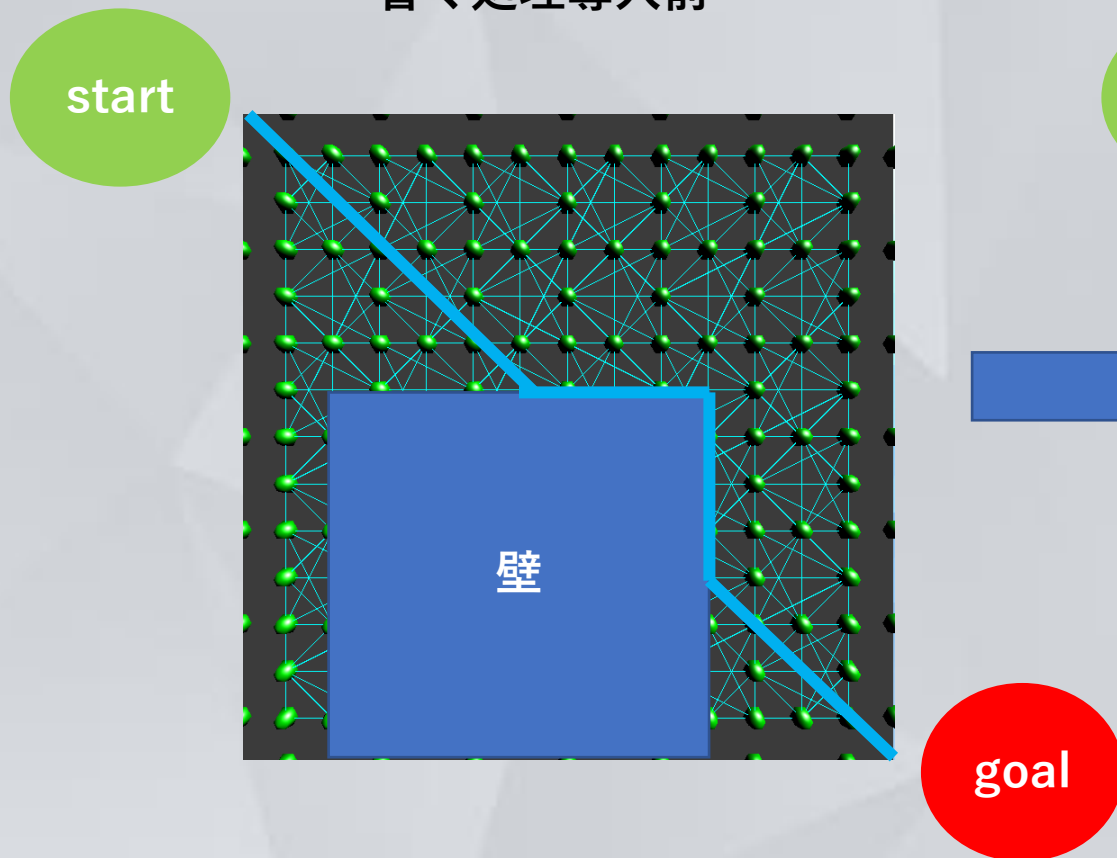
if (doConnect)
{
    // 頂点ABで構成されたポリゴン (以降ベースポリゴン1、2とすると) のエッジ4つで成り立つポリゴン
    // がベースポリゴン1または2の法線と方向が一致しなかったら、接続しない(一致したら接続)
    // モデルのポリゴンを貫通した接続を回避するため

    // 接続相手に同じ頂点を持っているポータルのナンバー(今生成された自身)を追加
    trPor.PushInitTargetPortalPointNumber(mePor.GetLocalNumber());

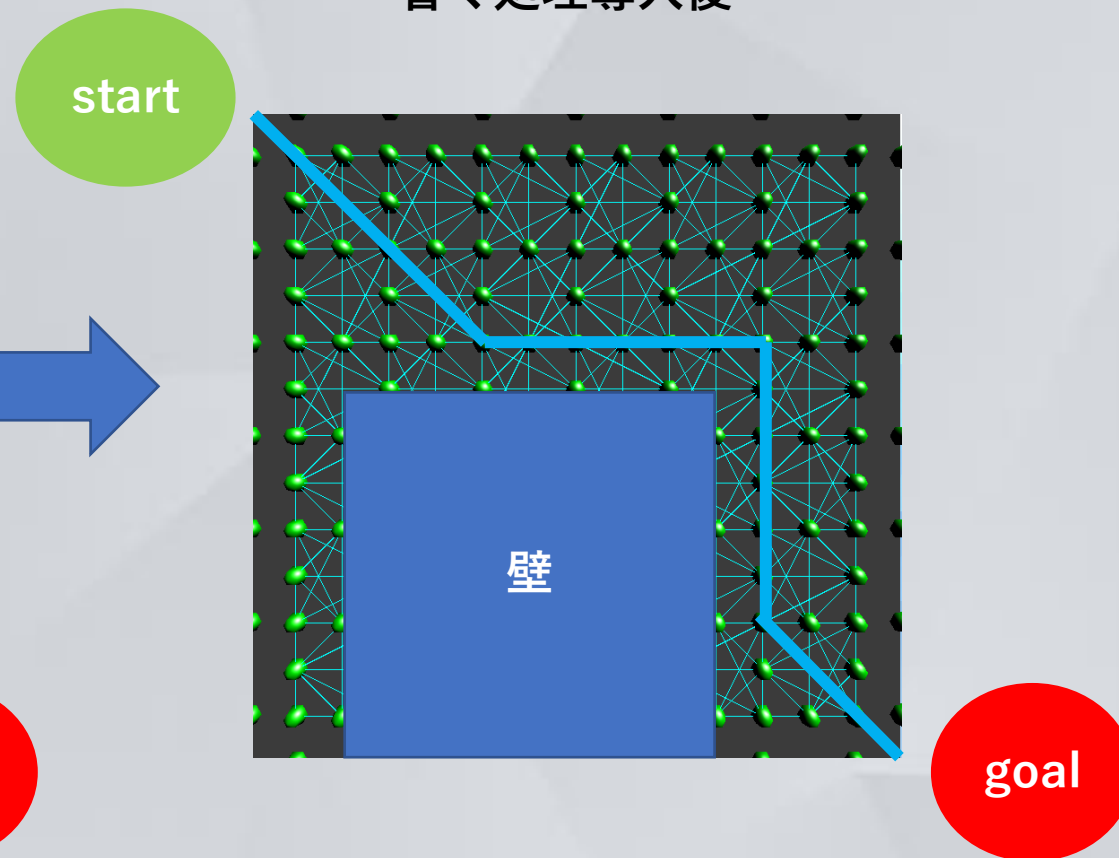
    // 今生成された自身に同じ頂点を持っているポータルのナンバー(相手)を追加
    mePor.PushInitTargetPortalPointNumber(trPor.second.GetLocalNumber());
}
```

ナビメッシュの構築と制御 [自然な移動をするための工夫]

省く処理導入前



省く処理導入後

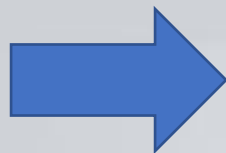
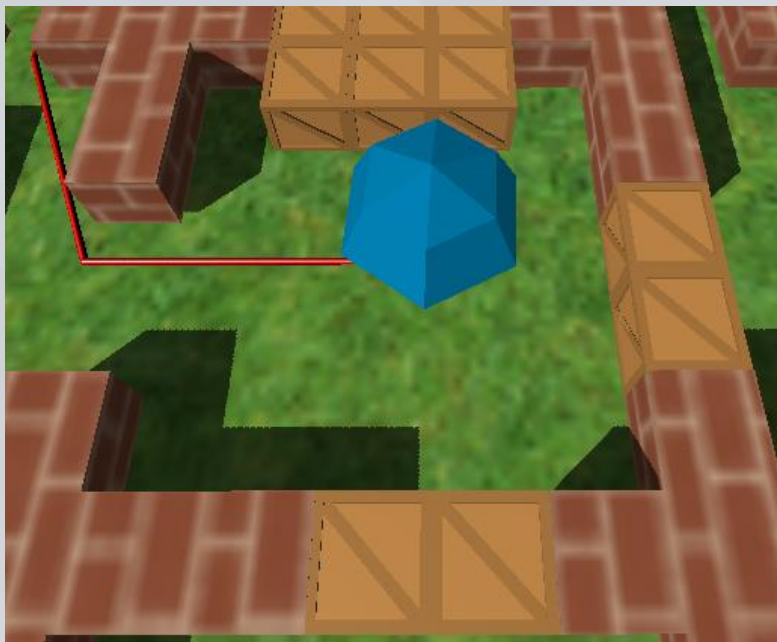


- ・ 壁に隣接したノードを通常ノードから省くことで、敵が経路探索するときに通らなくなり、より自然な経路で探索できるようにしました。

敵ルートのコスト算出

敵ルートのコスト算出

壁の設置変更前



壁の設置変更後



- ・ルートを構築する際に、遮断物に対する破壊して進むコストと、迂回して進むコストを比べて、敵のルートが**ステージ状況に合わせて、最適なルートを動的に構築**できるようにしました。

敵ルートのコスト算出

実コストの算出

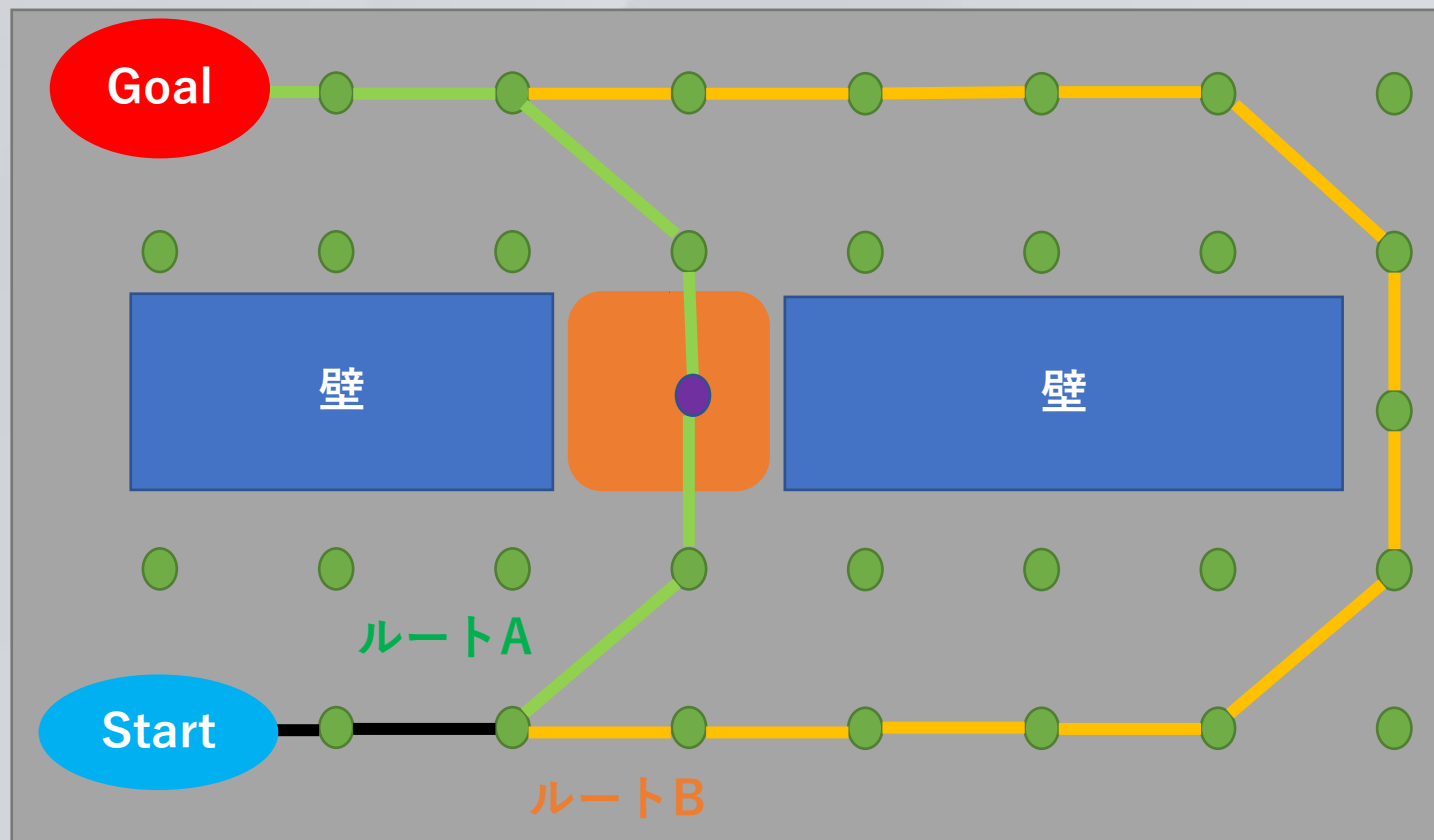
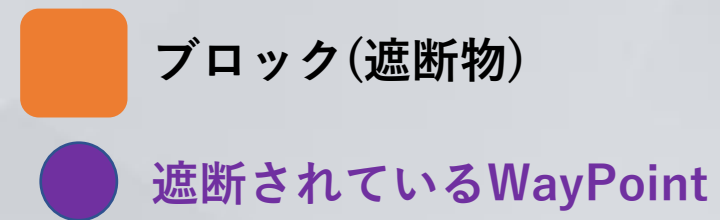
遮断されているWayPointのコスト

破壊コスト = ブロックHP / 敵のDPS

遮断されているWayPointコスト =
遮断される前のWayPointコスト + 破壊コスト

コストの比較

ルートA < ルートB



- ・ ルートAとルートBの実コストを算出し、コストがより低い方をルートとして、使用するようになりました。

処理負荷軽減の取り組み

処理負荷軽減の取り組み [実装結果]

軽減前

AppUpdate(); ≤ 15,984 ミリ秒経過



軽減後

AppUpdate(); ≤ 28 ミリ秒経過

- ・ ステージ変化時にノードの更新での処理負荷がありましたが、改善をしました。

当初：更新処理の経過時間 15,984 ミリ秒



現在：更新処理の経過時間 28 ミリ秒

処理負荷軽減の取り組み [設置時のみのOBB当たり判定]

- 1 設置物が設置または削除され、変化した範囲を記録します。
- 2 **変化した範囲**とノードで当たり判定します。
＞設置モデルはBOXとして考えOBBと点の当たり判定を行います。



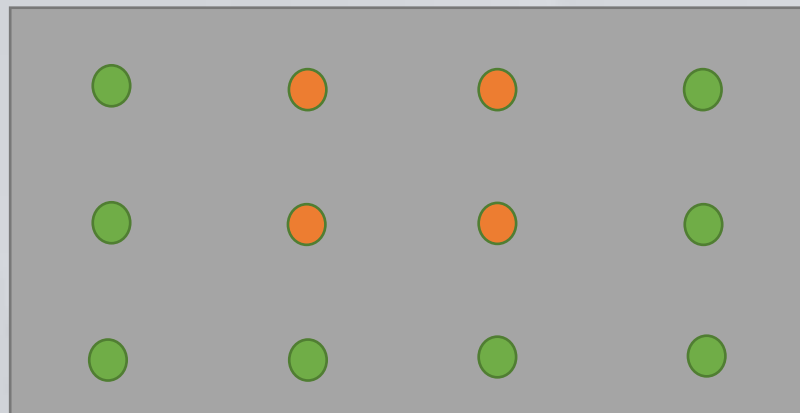
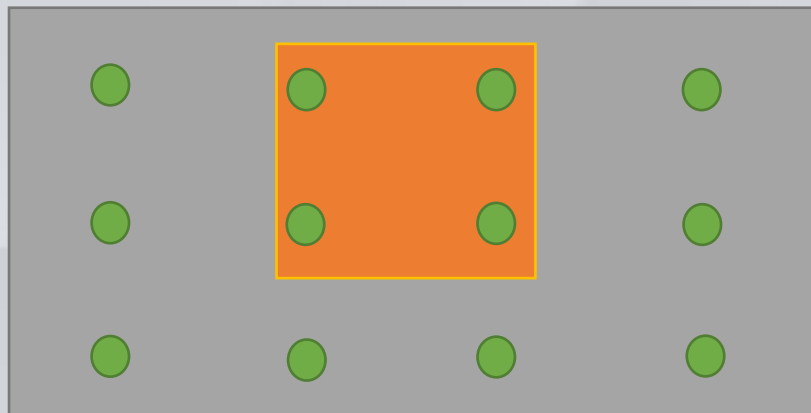
ノード



変化した範囲



当たり判定で当たったノード



```
// VECTORの座標のメンバー一つを見る
for (int m = 0; m < 3; m++)
{
    // 辺の半分の長さが0以下だったら計算しない
    if (GetVMem(halfLen, m) <= 0)
        continue;

    // 内積を使って四角形に対して現在座標の位置を割り出す
    float s = VDot(DISTANCE, GetVDirection(m, blockRot)) / GetVMem(halfLen, m);
    s = fabsf(s);

    // 四角形の辺からはみ出ている
    if (s > 1)
    {
        // どれだけはみ出したかの算出
        returnVec += (GetVDirection(m, blockRot) * (s - 1)) * GetVMem(halfLen, m);
    }
}

// NavPointの半径よりも近かったらtrue遠かったらfalse
// NavPointは点なのでここでは半径を0.5とする
return VSize(returnVec) < NAV_POINT_COLL_RADIUS_SIZE;
```

処理負荷軽減の取り組み [設置時のみのOBB当たり判定]

3 当たり判定で当たったノードを再生成または削除します。

- > 一部分のみのナビメッシュの更新で済むので、高速な再生成ができます。
- > 全フレームで回さなくて済むので、高速に処理できます。
- > モデルをOBBとして当たり判定することで、ポリゴン情報を扱った当たり判定を省けるので、高速に処理できます。

```
// ステージの変化した範囲を記録したコンテナに要素がなかったら
if (stageObjectChangedArea.empty())
    return;

for (auto altr = stageObjectChangedArea.begin(); altr != stageObjectChangedArea.end(); )
{
    // ステージオブジェクトがどう変化したかの種類
    STAGE_OBJECT_CHANGED_KIND changeKind = (STAGE_OBJECT_CHANGED_KIND)altr->first;
    // ステージオブジェクトのトランスフォーム
    Transform aTr = altr->second;

    for (auto nltr = navPosList.begin(); nltr != navPosList.end(); nltr++)
    {
        VECTOR3 nPos = nltr->second->GetPosition(); // NavPointの座標

        // NavPoint座標とステージオブジェクトが当たっていたら
        if (OBBToNavPointColl(aTr, nPos))
        {

```

※ステージが変化しない限り、当たり判定を行わないようにしています。

処理負荷軽減の取り組み [再計算をしなくていいようにする工夫]

1 ステージ配置物が設置された際

> OBBの当たり判定で、設置物に埋もれている判定を受けたノードを使えないノード配列に移動させます。

使えるノード配列

```
// 経路探索に使える状態のNavPointを管理するコンテナ  
std::unordered_map<int, NavPointBase*> activeNavPointList;
```

経路探索に使用

設置物に埋もれて
使えなくなった
ノード

使えないノード配列

```
// 経路探索に使えない状態のNavPointを管理するコンテナ  
std::unordered_map<int, NavPointBase*> inactiveNavPointList;
```

処理負荷軽減の取り組み [再計算をしなくていいようにする工夫]

2 ステージの配置物が削除された際

＞設置物が削除され、埋もれなくなったノードを、使えないノード配列から使えるノード配列に移動させます。

再計算せずに配列移動だけでなので処理負荷が低くて済みます。

使えるノード配列

```
// 経路探索に使える状態のNavPointを管理するコンテナ  
std::unordered_map<int, NavPointBase*> activeNavPointList;
```

経路探索に使用

設置物が削除され
て使えるように
なったノード

使えないノード配列

```
// 経路探索に使えない状態のNavPointを管理するコンテナ  
std::unordered_map<int, NavPointBase*> inactiveNavPointList;
```

JSONを使ったデータの外部化

JSONを使ったデータの外部化 [銃データ]

銃データの管理

- ・初めは、銃データを右のようにソース内で初期化していました。

- ・しかし、この場合ですと、データ変更の度にソースを変更する必要があり、作業に多くの時間を要していました。



- ・そこで、**銃データを外部化**することにしました。

```
float bulletSpeed          = 2500.0f;           // 弾の速度
VECTOR3 bulletVelocityFriction = VZero;        // 弾のウェロシティの抵抗値
bool isBulletGravity       = false;            // 弾が重力の影響を受けるかどうか
float range                = 1000.0f;          // 射程距離
float bulletSize           = 5.0f;            // 弾のサイズ 直径
float bulletLifeTime       = 8.0f;            // 弾の生存時間
std::unordered_map<COLLISION_OBJECT_KIND, float> bulletTargetDamageList = // 弾を当て相手と与えるダメージ量を設定
{
    { COLLISION_OBJECT_KIND::GROUND_BLOCK, 0.0f },
    { COLLISION_OBJECT_KIND::CORE_BLOCK, 0.0f },
    { COLLISION_OBJECT_KIND::WALL_BLOCK, 0.0f },
    { COLLISION_OBJECT_KIND::ENEMY, 10.0f },
};
std::set<SummonStageObjectInfo::SummonStageObjectData> bulletHitPosSummonStageObjectKindList = {}; // 着弾点に召喚するステージオブジェクトの種類コンテナ
std::set<Sound_ID::SOUND_ID> bulletHitPosSoundIDList = {}; // 着弾時のサウンド

EffekseerObjectManager::EFFEKSEER_KIND bulletHitPosEffectKind = EffekseerObjectManager::EFFEKSEER_KIND::EF_NONE; // 着弾のエフェクト
int ammo = 1000000; // 現在のマガジン内の弾数
int holdBullet = 100; // 弾の保持数
int magazineSize = 1000000; // 1マガジンのサイズ
float fireRate = 0.7f; // 連射速度 数値が大きいほど連射できる
float reloadTime = 3.0f; // リロード時間
float spread = 0.0f; // 拡散量 ショットガンなどの場合、数値が大きいほど拡散する
int palette = 1; // パレット数 ショットガンなどの場合複数同時発射なので、この数を増やす
EffekseerObjectManager::EFFEKSEER_KIND fireEffectKind = EffekseerObjectManager::EFFEKSEER_KIND::EF_NONE; // 発射エフェクト種類
std::set<Sound_ID::SOUND_ID> shotSoundIDList = // 発射音のサウンドIDコンテナ
{
    { Sound_ID::SOUND_ID::NORMAL_BULLET_SHOT_SE }
};

BulletInfo::BulletData bulletData =
    BulletInfo::BulletData
    (
        transform,
        bulletSpeed,
        bulletVelocityFriction,
        isBulletGravity,
        bulletSize,
        range,
        bulletLifeTime,
        bulletTargetDamageList,
        bulletHitPosSummonStageObjectKindList,
        bulletHitPosSoundIDList,
        bulletHitPosEffectKind
    );
```

JSONを使ったデータの外部化 [銃データ]

- 銃のデータをGunDataとして、JSONで外部化して管理するように実装しました。
- このデータをGunクラスで管理し、銃を使うPlayerクラスやTurretクラスで生成し保持することで、**どんなクラスでも銃を使用することが可能**になりました。

JSON情報

```
"Ammo": 15,  
"BulletData": {  
  "Friction": {  
    "posX": 0.0,  
    "posY": 0.0,  
    "posZ": 0.0  
  },  
  "HitPosEffectKind": -1,  
  "HitPosSoundIDList": [],  
  "HitPosSummonStageObjectList": [],  
  "IsGravity": false,  
  "LifeTime": 8.0,  
  "Range": 1000.0,  
  "Size": 25.0,  
  "Speed": 11000.0,  
  "TargetDataList": [  
    [  
      1,  
      0.0  
    ],  
    [  
      4,  
      0.0  
    ],  
    [  
      3,  
      0.0  
    ],  
    [  
      13,  
      10.0  
    ]  
  ],  
},  
"FireEffectKind": -1,  
"FireRate": 0.5,  
"HoldBullet": 1000000,  
"MagazineSize": 15,  
"Palette": 1,  
"ReloadTime": 0.0,  
"ShotSoundIDList": [  
  3013  
,  
,  
],  
"Spread": 0.0
```

データの種類

マガジン弾の数
弾の摩擦
ヒットエフェクト
ヒットサウンド
ヒットしたところに出すオブジェクト
弾の重力フラグ
弾の生存時間
弾の飛距離
弾の直径
弾のスピード
弾が当たる相手と与えるダメージ
発射エフェクト
発射間隔
保持弾数
マガジンサイズ
パレット数
リロード時間
発射サウンド
拡散値

JSONを使ったデータの外部化 [銃データ]

ImGuiでのデータ変更前



ImGuiでのデータ変更後



ゲーム内でImGuiを使ってデータを変更し、JSONへのロードとセーブ、
現在存在する銃クラスのインスタンスへの動的な適応を導入することで、
ゲームバランスの調整が容易になりました。

JSONを使ったデータの外部化 [敵データ]

敵データの管理

- ・ 敵の情報をEnemyDataとして、JSONの外部化を行いました。
- ・ このJSONの情報をゲーム内で、動的に変更し、敵キャラクターに適用することで、ソースを変更することなく、ゲームバランスの調整ができるようにしました。

JSONの情報

```
"AttackData": {  
  "AttackInterval": 2.0,  
  "AttackReach": 190.0  
},  
"CollisionData": {  
  "AttackCollTargetKindList": [...]  
},  
"BodyCollTargetKindList": [...]  
},  
"DropCoinData": {  
  "DropNum": 1,  
  "Probability": 100  
},  
"HPMax": 60.0,  
"ModelSizeData": {  
  "ModelScale": {  
    "posX": 2.0,  
    "posY": 2.0,  
    "posZ": 2.0  
  },  
  "ModelSize": {  
    "posX": 100.0,  
    "posY": 100.0,  
    "posZ": 100.0  
  }  
},  
"MoveData": {  
  "ClimbSpeed": 0.0,  
  "WalkSpeed": 150.0  
},  
"NavigationData": {  
  "NavigationMode": 1,  
  "NavigationTargetKind": 0,  
  "NodeUseKind": 0  
}
```

データの種類

攻撃のデータ
当たり判定を行う相手
ドロップするコインのデータ
HP
モデルサイズ
移動速度
ナビゲーションデータ

JSONを使ったデータの外部化 [ウェーブデータ]

JSONの情報

```
"WaveDataList": [  
  {  
    "EnemyKind": 0,  
    "SummonPosition": {  
      "posX": -1018,  
      "posY": 72,  
      "posZ": 992  
    }  
  },  
  ...  
]
```

- ・ ウェーブの情報をWaveDataとしてJSONで外部化して管理するように実装しました。



データの種類

召喚する敵の種類
召喚する敵の座標

JSONのファイル構成

名前

wave_01.json
wave_02.json
wave_03.json
wave_04.json
wave_05.json

- ・ ステージごとに、ウェーブのファイルを分割して管理しています。

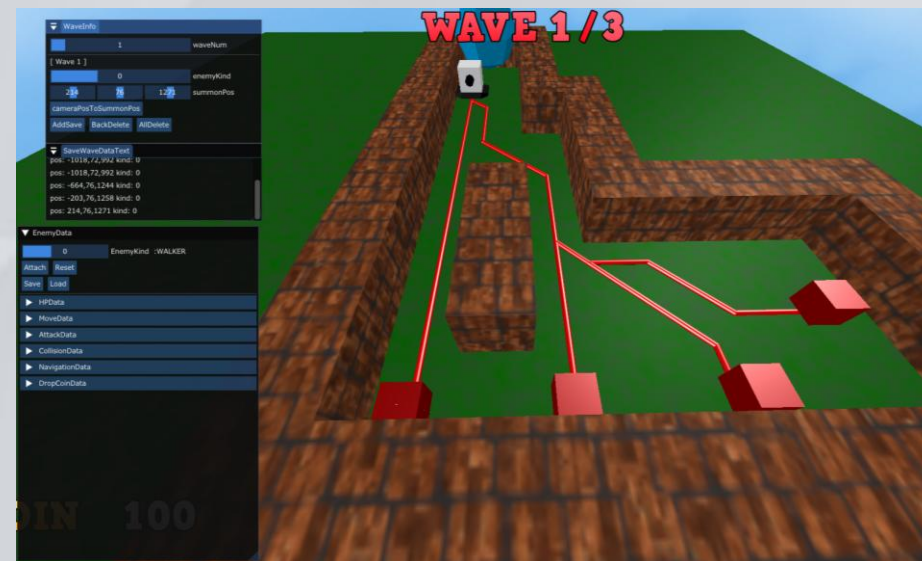
JSONを使ったデータの外部化 [ウェーブデータ]

敵ウェーブデータの管理

ImGuiでのデータ変更前



ImGuiでのデータ変更後



- ・ ウェーブ情報をImGuiを使い、ゲーム内で変更できるようにしたことで、敵ウェーブによる **ゲーム難易度の調整がしやすい** ように実装しました。

参考文献

- 1 Robotic Motion Planning: A* and D* Search
https://www.cs.cmu.edu/~motionplanning/lecture/AppH-astar-dstar_howie.pdf
- 2 Havok AI : ダイナミックなゲーム環境に応える経路探索
https://cedil.cesa.or.jp/cedil_sessions/view/1643
- 3 『LEFT ALIVE』における地形表現とナビゲーションAI
https://cedil.cesa.or.jp/cedil_sessions/view/2065